Comparative Study of PID and State-Space Pole placement Control for a Differential Drive Mobile Robot Progressive Modeling and Simulation

Mohammed Emad Said Soliman Mohammed
University of Debrecen / faculty of engineering
Department of Mechatronics (UHCI4H)
Debrecen , +36205221083
mohammed.emad@mailbox.unideb.hu

Supervisor: Kis Károly Árpád
Department of Electrical Engineering and Mechatronics
University of Debrecen
Debrecen, Hungary
kis.karoly@eng.unideb.hu

Dr, Hajdu Sandor
Department of Electrical Engineering and Mechatronics
University of Debrecen
Debrecen, Hungary
sandor.hajdu@eng.unideb.hu

***Abstract*— In this report, the mathematical model and simulation of a line-following control architecture of a differential drive mobile robot (DDMR) [1] is presented. It is a comparison of two closed-loop control strategies, namely, a decoupled Proportional-Integral-Derivative (PID) controller [7] and a State-Space pole placement controller with a Luenberger observer [5]. These Models are first mathematically validated and then combined with 3D Simscape Multibody [6]. The simulation have real word limitations such as cascaded DC motor dynamics, non-ideal battery models, and H-bridge driver circuits. The results show that PID control are better in idealized situations, but the State-Space pole placement have better stability, and higher forward velocity retention under complex limitations.**

Keywords— Differential drive mobile robot, path tracking, PID control, state-space control, pole placement, Luenberger observer, Simscape Multibody, cascaded control.

I. INTRODUCTION

One of the basic problems in mobile robotics is the gap between ideal mathematical models and real physical systems [2]. Differential drive mobile robots (DDMRs) have many non-linearities, including chassis inertia and electrical actuation lag. Classical PID methods are still widely used in the industry because they are easy to tune [7] but they can be affected by coupled physical constraints. The report presents the development and testing of sophisticated control architectures for a line following DDMR, culminating in a realistic electromechanical simulation. The project aims to develop a verified 3-Degree of Freedom (3-DoF) kinematic baseline [3] for the evaluation of two parallel control schemes: classical PID [4] and modern State-Space pole placement with full-state feedback [1].The controllers are then tested in a 3D physical plant with Simscape Multibody [6] to rigorously test these algorithms with a on-ideal batteries, and the H-bridge circuits together.

II. KINEMATIC MODELING OF THE ROBOT

Differential Drive mobile robot has two wheels that have their own drive motors, but are connected to each other by a common lateral axis. The kinematic model is a robot motion model that considers the robot's motion in a global reference frame and assumes that the robot has no longitudinal or lateral slip, only pure rolling [1], [2]. Suppose that R is the radius of the wheels and L is the width of the track, which is the distance between the two drive wheels. The angular velocities of the left wheel, ω_L and right wheel, ω_R are proportional to their linear velocities, v_L and v_R respectively:

$$v_L = R \cdot \omega_L \tag{1}$$

$$v_R = R \cdot \omega_R \tag{2}$$

The robot's overall linear velocity v is the average of the velocities of the individual wheels, and the robot's angular velocity ω is the difference of the velocities of the individual wheels:

$$v = \frac{v_R + v_L}{2} = \frac{R(\omega_R + \omega_L)}{2} \tag{3}$$

$$\omega = \frac{v_R - v_L}{L} = \frac{R(\omega_R - \omega_L)}{L} \tag{4}$$

The state variables for the robot's posture in a 2D Cartesian coordinate system are the position variables (x, y) and the angle of orientation θ. The rates of change (time derivatives) of these state variables are written as a function of the center velocities:

$$\dot{x} = v \cdot \cos(\theta) = \frac{R}{2}(\omega_R + \omega_L)\cos(\theta) \tag{5}$$

$$\dot{y} = v \cdot \sin(\theta) = \frac{R}{2}(\omega_R + \omega_L)\sin(\theta) \tag{6}$$

$$\dot{\theta} = \omega = \frac{R}{L}(\omega_R - \omega_L) \tag{7}$$

To adapt these kinematic relationships in the control system , the equations have first to be formatted to the matrix form :

$$\begin{vmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{vmatrix} = \begin{bmatrix} \frac{R}{2}\cos(\theta) & \frac{R}{2}\cos(\theta) \\ \frac{R}{2}\sin(\theta) & \frac{R}{2}\sin(\theta) \\ \frac{R}{L} & -\frac{R}{L} \end{bmatrix} \begin{bmatrix} \omega_R \\ \omega_L \end{bmatrix} \tag{8}$$

The formulation shows that the system is equivalent to the standard Jacobian matrix of a unicycle-type non-holonomic mobile robot, but scaled by the physical parameters of the wheels' radius and track width [3].

Finally, the continuous position and orientation of the robot over time $t$is obtained by integrating the kinematic rates from their initial values $(x_0, y_0, and\, \theta_0)$

$$\theta(t) = \theta_0 + \int_0^t \omega(\tau)d\tau \tag{8}$$

$$x(t) = x_0 + \int_0^t v(\tau)\cos(\theta(\tau))d\tau) \tag{9}$$

$$y(t) = y_0 + \int_0^t v(\tau)\sin(\theta(\tau))d\tau) \tag{10}$$

The derived mathematical model was implemented and validated in MATLAB and Simulink to ensure accuracy of the theory as shown in figure 1 . A custom kinematic block was designed with the standard operational blocks: trigonometric functions, gains corresponding to the R and L parameters and integrators to compute the continuous state variables from the input angular velocities.

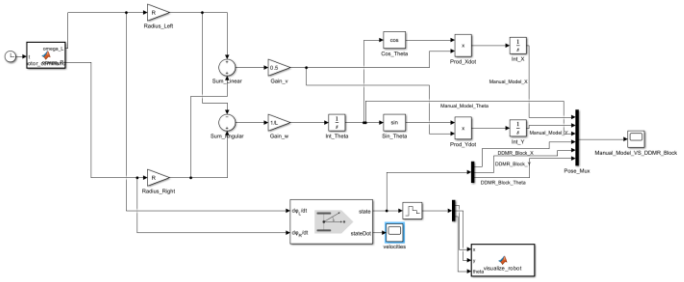


Figure 1 showing the kinematic modeling of the robot on MATLAB

The results of this manual model were compared directly to the built-in "Differential Drive Kinematic Model" block available in MATLAB to validate the manual implementation. The same test input was applied to both the manual mathematical model and the standard MATLAB block. The graphical output shows that there is a complete match between the signals obtained from both models as shown in figure 2 laying a good basis for the future work on the model .

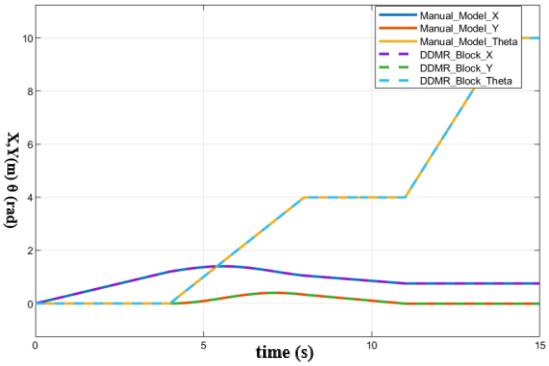


Figure 2 showing the output comparison between the built-in block and the manual model

III. THE FORMULATION AND SIMULATION OF A PID CONTROLLER FOR LINE FOLLOWING.

A. Controller design and formulation

feedback control system was designed to control the heading and forward velocity of the robot to track the line and measure the track error in real time to provide accurate line following. The main input error to the system from the 6 sensor array is a lateral deviation error $e(t)$, which is the distance between the robot center and the line . An effective control scheme is formed by the use of two PID controllers in parallel. This is a decoupled architecture that makes the non-holonomic control problem easier; one controller controls the forward linear velocity v, the other controls the angular steering velocity ω [4], [7].

1) Heading Control: The rotational control loop is used to compute the error, and continuously correct the angular velocity to follow the track:

$$\omega(t) = K_{p,\omega}e(t) + K_{i,\omega}\int_0^t e(\tau)d\tau + K_{d,\omega}\frac{de(t)}{dt} \qquad (11)$$

2) Forward Speed Control: The parallel speed controller is also adaptive to decrease the forward speed in proportion to the size of the lateral error during sharp turns to maintain dynamic stability:

$$v(t) = 0.4 - \left(K_{p,v}|e(t)| + K_{i,v}\int_0^t |e(\tau)|d\tau + K_{d,v}\frac{d|e(t)|}{dt}\right) \qquad (12)$$

This sets the maximum nominal speed to 0.4 m/s, and actively slows down when the absolute value of the track error $|e(t)|$grows.

3) Inverse Kinematics : Both parallel controllers produce v and ω values which are directly mapped to the drive wheels by inverse kinematics:

$$\omega_R(t) = \frac{v(t) + \frac{L}{2}\omega(t)}{R} \qquad (14) \qquad \omega_L(t) = \frac{v(t) - \frac{L}{2}\omega(t)}{R} \qquad (15)$$

B. Simulink Implementation

The closed-loop parallel control architecture was developed using MATLAB and Simulink as shown in figure 3 . A MATLAB function block models the 6-sensor array to simulate the error between the robot and the pre-defined figure-eight track points. The raw error is fed directly to the heading PID controller and the absolute error is fed directly to the speed PID controller. The outputs of their linear velocity and angular velocity are then fed into the inverse kinematics block to calculate the velocity of each wheel. These velocities are fed continuously to kinematic state integrators, updating the robot

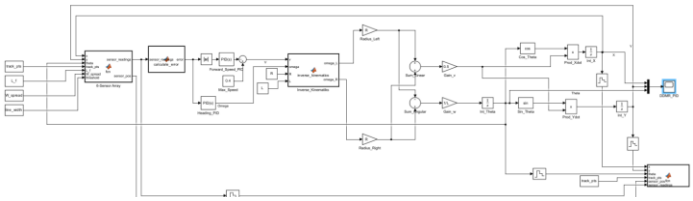


Figure 3showing the parallel PID controller design

C. Simulation Results

The system was tested in a 30 second simulation on a figure 8 benchmark track. The resulting state trajectories (x, y and θ) illustrate the robot's ability to follow the overall geometry of the path, which is the crossover geometry. The plotted signals, however, show that there are significant high frequency fluctuations, or “chatter,” on the paths as shown in figure 4. The wobble here is an indication that the robot is still on the line, but that the control gains or sensor logic at the moment generates too much micro-correction. A more detailed numerical analysis of these deviations and further tuning of the controller will be carried out in future work to smooth the system's dynamic response. The PID gain values for the system are (5,0,0) for the kp, ki and kd in order for the breaking controller and (50, .0001,0) for the heading controller , the values are manually tuned for speed but should be retuned using Ziegler Nichols method in the near future for better system behavior

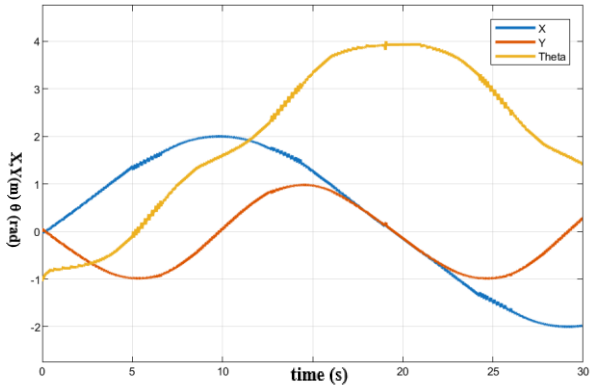


Figure 4 showing the PID system response to the 8-figure track

IV. STATE-SPACE CONTROL AND POLE PLACEMENT

The kinematics of the system were reformulated in a state space model to develop a more solid modern control architecture for line following. This method enables the state of the system to be fed back throughout the state and the dynamic response of the system to be manipulated exactly by pole placement [1].

A. System State and Control Input:

$$x = \begin{bmatrix} |e_y| \\ e_y \\ e_\theta \end{bmatrix} \quad u = \begin{bmatrix} \Delta v \\ \omega \end{bmatrix} \tag{16}$$

B. The Full 3-DoF Kinematic Model and Decoupling.

The complete kinematic state-space model is given by:

$$\dot{x}_{full}(t) = A_{full}x_{full}(t) + B_{full}u_{full}(t) \tag{17}$$

Where the system matrices are defined as:

$$A_{full} = \begin{bmatrix} 0 & -V_{ref} & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, B_{full} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}, C_{full} = [0 \quad 1 \quad 0] \tag{18}$$

This full model is decoupled into a braking system and a steering system to ease the control design. The braking subsystem is just one that is defined by the values of $A_{brake} = 0$, $B_{brake} = 1$. The steering subsystem is decoupled and controlled by:

$$\dot{x}_{steer}(t) = A_{steer}x_{steer}(t) + B_{steer}\omega(t) \tag{19}$$

$$y(t) = C_{steer}x_{steer}(t) \tag{20}$$

The state-space matrices for this steering subsystem are obtained as:

$$A_{steer} = \begin{bmatrix} 0 & -V_{ref} \\ 0 & 0 \end{bmatrix}, \quad B_{steer} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad C_{steer} = [1 \quad 0] \tag{21}$$

C. Full-State Estimation and Pole Placement

A Luenberger Observer can be designed to estimate the entire steering state $\hat{x}_{steer}(t)$ which contains both the lateral error $\hat{e}_y$ and the unmeasured heading error $\hat{e}_\theta$ [5]. The dynamics of the observer are defined as:

$$\dot{\hat{x}}_{steer}(t) = A_{steer}\hat{x}_{steer}(t) + B_{steer}\omega(t) + L_{obs}\big(y(t) - \hat{y}(t)\big) \tag{22}$$

The full state is estimated and a state feedback control law is implemented using the pole placement method. The control vector is organized in the following way:

$$X_{ctrl} = \begin{bmatrix} |e_y| \\ \hat{e}_y \\ \hat{e}_\theta \end{bmatrix} \tag{23}$$

This vector is multiplied by the state-feedback gain matrix K to create the control inputs:

$$u = -KX_{ctrl} \tag{24}$$

The final K matrix is :

$$K = \begin{bmatrix} K_{brake} & 0 & 0 \\ 0 & K_{steer,1} & K_{steer,2} \end{bmatrix} \tag{25}$$

This leaves the final decoupled control equations, for the forward linear velocity v and the angular steering velocity ω:

$$v(t) = V_{ref} - K_{brake}|e_y(t)| \tag{26}$$

$$\omega(t) = -K_{steer,1}\hat{e}_y(t) - K_{steer,2}\hat{e}_\theta(t) \tag{27}$$

D. Simulink Implementation

The state space controller and Luenberger observer were designed and coded in MATLAB and Simulink to control the previously defined inverse kinematics. The whole state-space pole placement was reduced to a matrix gain block (the K matrix) with the state errors as inputs. The respective gain matrices A_{steer}, B_{steer}, and L_{obs}, are mapped directly to the corresponding standard addition and multiplication blocks as show in figure 5 . for the pole values [-.9, -1.89] have been chosen and will be adjusted in the future for better behavior

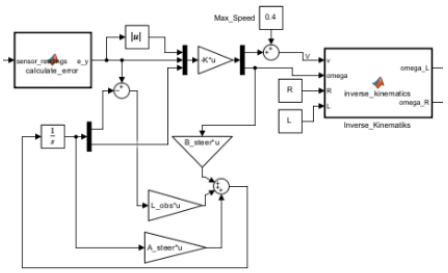


Figure 5 showing the SS model and the observer model Arcature

E. Simulation Results

The state-space control system was evaluated over the same figure-eight track for a 30 second simulation. The resulting trajectories (x, y, and θ) illustrate that the system was able to successfully complete the line-following task and made minimal overall deviation from the desired path when navigating the geometry of the crossover as shown in figure 6. However, one can see that the robot's motion is slightly wobbly and wonky from the previous PID implementation. High frequency chatter is clearly apparent. This wobble will need to

be corrected with further detailed numerical analysis and gain tuning

V. PHYSICAL MODELING IN SIMSCAPE

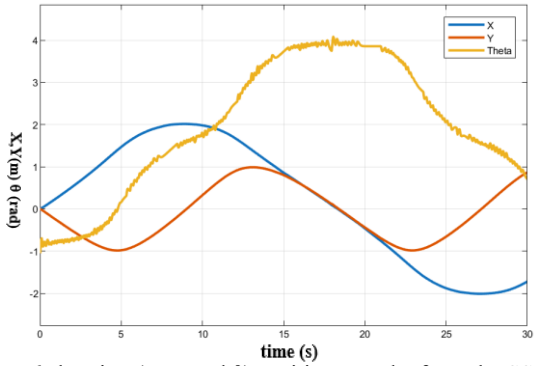


Figure 6 showing (x, y, and θ) positions results from the SS pole placement model as an output of the 8-figure track

MULTIBODY

The simulation was extended beyond a purely mathematical simulation by building a 3D physical model of the differential drive robot in the Simscape Multibody environment. This physical representation includes rigid body mechanics, inertial properties, joint constraints and contact forces, to simulate realistic real world physics[6].

The physical model is based on a central solid block representing the chassis with a mass of 0.5 kg that connects to a defined global reference frame and solid ground as shown in figure 7. The three wheel subsystems attached to this main chassis are a left drive wheel and a right drive wheel, both 20 g, and a passive spherical caster wheel of 30 g to ensure a multi-point structure stability.

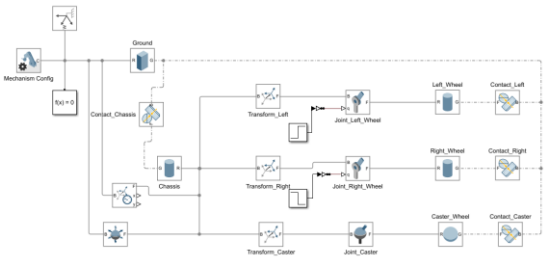


Figure 7 showing the simscape model for the differential drive mobile robot

In order to enable the left and right drive wheels to rotate independently about their own axes when being driven by input signals, revolute joint blocks are then applied. Lastly, spatial contact force blocks are applied between each wheel part and the ground block, to represent contact forces between the surfaces. These contact blocks were calibrated by tuning the contact stiffness and damping parameters to accurately represent physical springiness of the wheels, and by using realistic friction coefficients to accurately model physical traction and rolling resistance.

The resulting 3D mechanical visualization is a simple cylindrical body with the two main drive wheels and the additional caster wheel as shown in figure 8 . This is a built-up multi-body structure that becomes the dynamic physical plant to be used for the next test and validation of the control algorithms designed previously. A full rendered video of the robot following the line is done and it shows the robot moving on the bath appearing on figure number 8

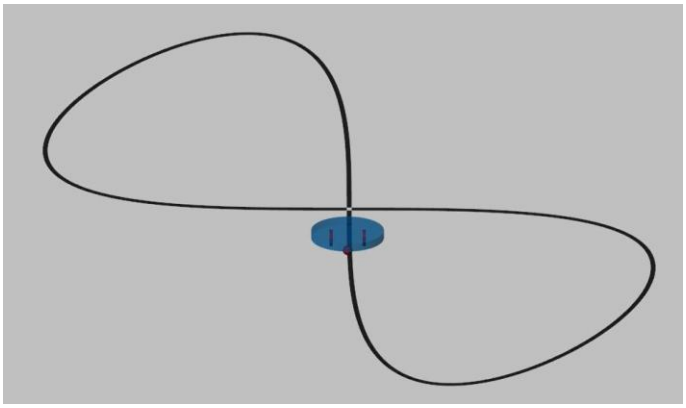


Figure 8 showing the generated physical model for the DDMR

VI. INTEGRATED PHYSICAL SIMULATION AND MOTOR CONTROL.

The control architectures were then connected to the Simscape Multibody physical plant and electrical actuation components to create a complete closed-loop system for rigorous testing of the line-following algorithms in a realistic environment.

A. The Actuation and Cascaded Control Architecture.

The Simscape model was coupled with models for the robot's electrical hardware to physically drive the model. A Simscape Multibody interface was used to connect the electrical and mechanical domains, with the simulated electrical motors directly connected to the physical revolute wheel joints of the chassis as shown in figure 9. This setup gives rise to a cascaded control structure: the line-following algorithms in the main control loop provide velocity references, and the dedicated PID motor controllers provide the inner loop to control the actual wheel speeds.

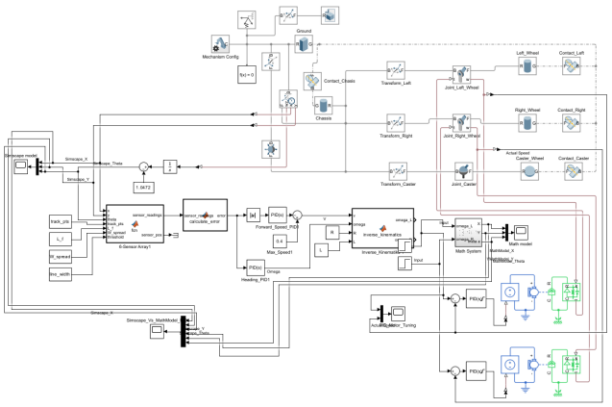


Figure 9 showing the simscape model attached to the PID control and actuated with motors

The inner loop was first calibrated separately before carrying out the full track simulation. The individual motor PID controllers were tuned using a step input block to ensure that the commanded wheel speeds are accurately achieved and

maintained. The step response graph for the resulting tuned inner-loop controllers shows that the controllers have reached the desired velocity with fast settling time and have very little overshoot as shown in figure 10.

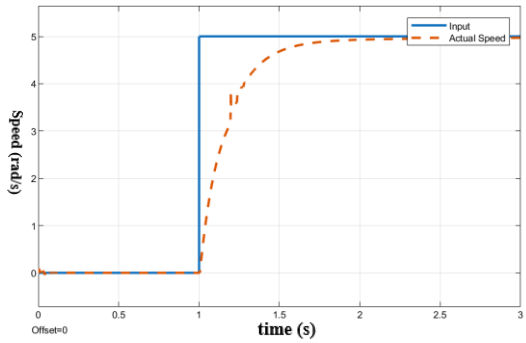


Figure 10 shwoing the response of the inner PID loop to a step input

B. PID Controller Integration and Simulation

The parallel PID controller was cascaded with the parallel architecture to test the performance of the parallel PID controller under physical constraints. The raw and absolute lateral errors from the simulated sensors were sent to the heading and forward speed PID blocks which continually corrected the velocity inputs to the inverse kinematics and the subsequent motor drivers.

The integrated PID system was simulated for a period of 30 s. The scope output showed very stable line following characteristics. The physical Simscape trajectories (plotted as solid lines) smoothly and accurately tracked the ideal MathModel trajectories (plotted as dashed lines) as shown in figure 11. The robot was able to cross the complicated geometry of the track without any erratic structural movements and with minimal deviation from the track.

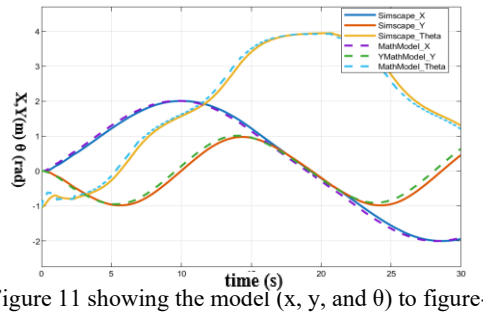


Figure 11 showing the model (x, y, and θ) to figure-8 controlled with PID and actuated with motor

C. State-Space Controller Integration and Simulation

After PID evaluation, the State-Space pole placement controller and the Luenberger observer were cascaded to the same physical system as shown in figure 12. The lateral deviation from the sensor array was used to calculate the full state error vector to be multiplied by the K gain matrix, thus generating the control signals for the motor loop.

The State-Space system was also simulated for 30 seconds. The physical robot was able to successfully complete the figure 8 path, although the dynamic response was less stable than the PID configuration as shown in figure 13. The resulting simulation plots show large deviations from the X position and Y position, as well as wobbly heading deviations, at high frequencies.

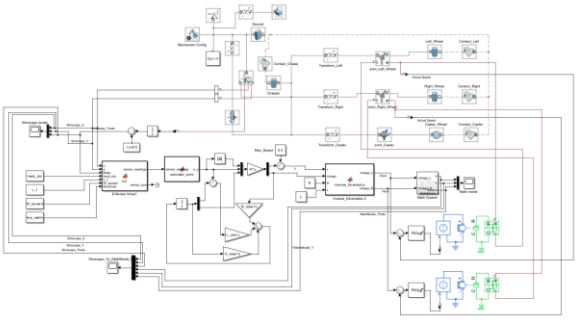


Figure 12 showing the State Space pole placment controlled robot actuiated by motors

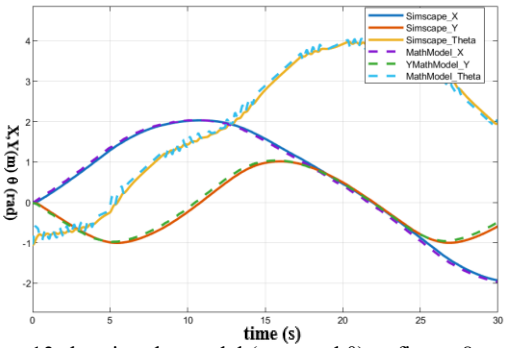


Figure 13 showing the model (x, y, and θ) to figure-8 controlled with SS pole placement and actuated with motor

VII. BATTERY AND H-BRIDGE INTEGRATION

In the previous simulation iterations, it was assumed that the power supply was an ideal power supply that can provide the required voltage to the DC motors instantly. A non-ideal battery model and H-bridge driver circuits were added to the inner control loop to include hardware constraints as closely as possible to those in real-life applications as shown in figure 14 . This addition added realistic electrical dynamics including voltage limits and Pulse Width Modulation (PWM) switching behaviors, which changed the dynamic response of the robot in a fundamental way. PID Controller: The PID architecture had difficulty with the newly added electrical inertia, and needed to be adjusted for both controllers so that the line would adhere. It was not able to track the line efficiently until its maximum forward velocity was lowered to 0.3 m/s. State-Space Controller: The maximum speed of the state-space controller remained the same as before. To reduce the driver lag, however,

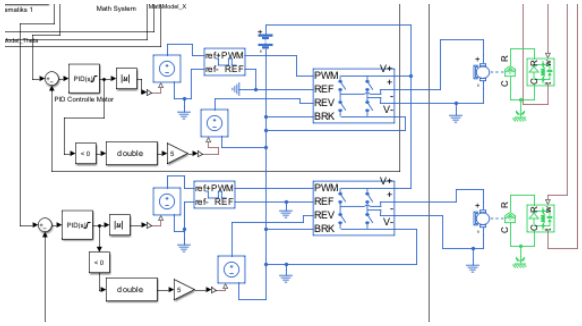


Figure 14 showing the addition of H-bridge and battery model to the exisitng model

the state-feedback gain matrix K was increased to produce a more active corrective action in response to the state errors. The new poles are [-4, -5] and will be adjusted in the suture again for better behaavior.

B. Comparative Results

When the system was simulated under these realistic constraints, the opposite comparative performance was obtained. The PID controller was still found to be somewhat unstable, although it was running at a slower speed. Its state trajectories showed significant wobble and chatter at high frequencies as it attempted to adjust to the physical driver dynamics. By comparison, the re-tuned State-Space controller was very stable and had excellent line following stability with much smoother trajectory plots. Moreover, the State-Space system was able to keep the forward velocity high, so that it actually went a lot farther along the figure-eight track in the same 30 seconds of simulation time than the PID model as shown in the figure 15 and 16. This result demonstrates that the complete-state feedback architecture is better suited for tackling complicated, practical electromechanical constraints.

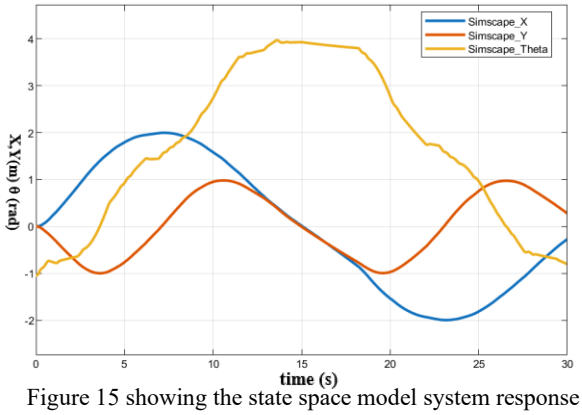


Figure 15 showing the state space model system response with the introduction of H-bridge and battery model

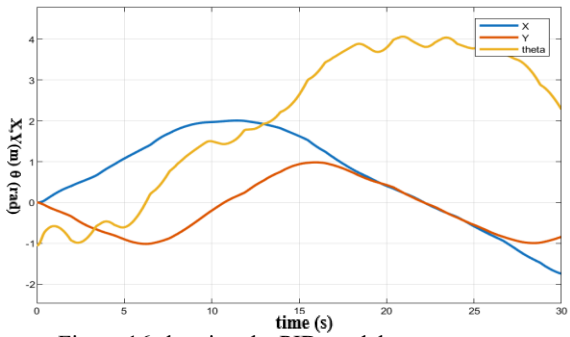


Figure 16 showing the PID model system response with the introduction of H-bridge and battery model

VIII. CONCLUSION

This report provided a detailed progressive development and simulation of a line-following control architecture for a differential drive mobile robot. The project was able to take the validated kinematic mathematical model as a starting point and develop through ideal control designs into a highly realistic 3D Simscape Multibody simulation. The decoupled PID controller was found to be stable under idealized theoretical conditions, but when physical constraints were added (chassis inertia, friction, non-ideal battery, and H-bridge driver dynamics), it was found to be unstable. In these realistic scenarios, it was found that the state-space pole placement controller, with full-state estimation provided by a Luenberger observer, was much more capable. It had greater forward velocities and better trajectory stability under the influence of complex coupled electromechanical dynamics. In the future work, the next step in this thesis project will be to perform detailed numerical analysis to optimize the state-space matrix gains, to remove any high-frequency chatter and to be ready to deploy the optimized control architecture onto the actual robot hardware.

IX. ACKNOWLEDGMENT

I would like to thank my supervisor, Károly Árpád Kis, for his invaluable guidance, patience and constant support during this work. Without his guidance this thesis would not have come to the present form.

References

[1] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. London, U.K.: Springer, 2009. doi: 10.1007/978-1-84628-642-1.

[2] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.

[3] G. Campion, G. Bastin, and B. D'Andrea-Novel, "Structural properties and classification of kinematic and dynamic models of wheeled mobile robots," *IEEE Trans. Robot. Autom.*, vol. 12, no. 1, pp. 47-62, Feb. 1996. doi: 10.1109/70.481750.

[4] Y. Kanayama, Y. Kimura, F. Miyazaki, and T. Noguchi, "A stable tracking control method for an autonomous mobile robot," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Cincinnati, OH, USA, 1990, pp. 384-389. doi: 10.1109/ROBOT.1990.126006.

[5] D. G. Luenberger, "An introduction to observers," *IEEE Trans. Autom. Control*, vol. 16, no. 6, pp. 596-602, Dec. 1971. doi: 10.1109/TAC.1971.1099826.

[6] MathWorks, "Simscape Multibody User's Guide," The MathWorks, Inc., Natick, MA, USA, 2026. [Online]. Available: https://www.mathworks.com/help/sm/

[7] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. Research Triangle Park, NC, USA: ISA, 1995.